DESIGN DOCUMENT: AI-POWERED REAL ESTATE PROPERTY VALUATION ASSISTA

1. Problem statement

Property valuation from an informal description is difficult to make consistent and explain: facts may be incomplete, comparable sales are hard to locate, and a single number hides uncertainty. This project addresses that workflow for historical Ames, Iowa homes by extracting known property facts, finding similar recorded sales, estimating a value, and presenting supporting evidence, uncertainty, feature impacts, safeguards, and optional quality evaluations. It is a decision-support prototype, not a professional appraisal or a current-market service.

2. Requirements and boundaries

The design supports natural-language and structured property input, comparable-property retrieval, price estimation, uncertainty communication, feature impact explanations, evaluation, and feedback. Numeric estimates come from the local regression pipeline or a clearly labelled fallback. LLM and evaluator integrations may assess explanation quality, but must not modify the numeric estimate. The system does not provide an appraisal, guarantee, or causal analysis.

3. System components

- React/Vite frontend: separate dashboard sections for overview, valuation, related properties, feature impacts, evidence, safeguards, explanation review, AI quality evaluation, and feedback.
- FastAPI service: typed request validation, valuation and retrieval routes, health/configuration information, feedback persistence, and evaluation endpoints.
- LangGraph workflow: shared valuation state is processed through property interpretation, retrieval, prediction, risk/safeguard analysis, and response assembly. Deterministic implementations remain available for optional integration failures.
- Hybrid retrieval: normalized local sentence-transformers/all-MiniLM-L6-v2 embeddings are stored in ChromaDB; rankbm25.BM25Okapi performs lexical search. Metadata filtering constrains candidates and reciprocal-rank fusion combines the semantic and lexical rankings.
- Prediction and explanation: candidate tree regressors are selected by holdout MAE. SHAP supplies feature impacts where available, with an explicitly labelled feature-importance fallback. Comparable evidence supports uncertainty and fallback estimates.
- Evaluation: optional Claude-based explanation judge and one DeepEval GEval metric,

Property Recommendation Quality. Evaluation status, score, verdict, and reason are displayed independently of numeric valuation.
- Persistence: local SQLite records valuation history, feedback, and evaluation data when available. Persistence failure does not disable valuation.

4. Architecture design

The architecture diagram shows the offline dataset/index-building path and the online request path. Offline, each source row is prepared into a canonical property record, rendered as one short text document, embedded, and stored in ChromaDB with structured metadata. Online, a user request reaches the API, flows through LangGraph, and the retrieval agent obtains evidence through filtered semantic and lexical search. Retrieved comparables ground the deterministic valuation and explanation. Optional Claude evaluation is separate from the numeric valuation. The API returns the response to the React dashboard.

The overall design separates data preparation, index creation, retrieval, regression, explanation, and quality evaluation. The Chroma store is a retrieval index, not the source of sale prices; the source CSV remains authoritative for training and rebuilding.

5. Technology selection

Technology	Role and reason for selection
Claude via Anthropic Messages API	Optional explanation-quality judge, selected because the project uses a configurable Claude model for natural-language evaluation. Claude is not the valuation model and does not create embeddings or revise calculated values. If unavailable or unconfigured, deterministic validation is returned.
sentence-transformers/all-MiniLM-L6-v2	Local sentence encoder for short property descriptions. It avoids a paid embedding API, can be cached and loaded once per process, and produces normalized vectors suitable for cosine similarity.
ChromaDB	Persistent local vector index with cosine distance and property metadata filters; suitable for rebuilding and running this single-project dataset locally.
rank-bm25 / BM25Okapi	Local lexical retrieval complements semantic similarity, especially for explicit counts, neighborhood names, and exact terms.
LangGraph	Expresses property interpretation, retrieval, valuation, risk, and decision stages as a typed shared-state workflow with explicit ordering.
FastAPI + Pydantic	Provides typed JSON APIs, request validation, generated OpenAPI documentation, and clear HTTP validation errors.
scikit-learn, pandas, NumPy	Tabular data preparation, candidate regression models, metrics, and numeric feature processing.
React + Vite	Builds the interactive dashboard and lets the user view valuation,

evidence, safeguards, and evaluations in separate sections. |
| SQLite | Lightweight local persistence for valuation history, feedback, and evaluations without requiring a database service. |

6. Retrieval strategy

Record preparation and chunking

The current source is structured tabular data, not long narrative documents. Preprocessing canonicalizes source column names and quality ratings. Each property row is rendered into one compact property description (neighborhood, bedroom/bathroom counts, area, year, quality, condition, and garage facts) and retained as one retrieval document. There is no text splitting or multi-chunk strategy in the current implementation: one record maps to one document, one vector, and one property ID. This keeps each result's facts attached to the correct sale record and avoids duplicate chunks from the same home. If long documents are added later, chunk size and overlap will need separate evaluation.

Embeddings and stores

The embedding model is sentence-transformers/all-MiniLM-L6-v2. It runs locally, is loaded once per process through a cache, and encodes batches with normalized vectors. ChromaDB stores vectors and metadata using cosine distance. A dataset and schema fingerprint detects when the local collection needs rebuilding. BM25 uses tokenized property descriptions as a separate lexical index.

Search and fusion

The query is first interpreted into structured property features and rendered as a comparable query document. Neighborhood, year, bedroom, bath, room, living-area, and lot-area filters are applied with progressive relaxation if too few candidates remain. Semantic ranking comes from ChromaDB; if ChromaDB fails, cached local MiniLM vectors are searched. Lexical ranking comes from BM25Okapi. Reciprocal-rank fusion (RRF, configured RRFK, default 60) combines available rank lists. A property similarity score and a small feedback prior then order the final comparable results. If MiniLM is unavailable, retrieval degrades to BM25-only. The response exposes the retrieval method and fallback state.

7. Agent design

LangGraph defines a sequential StateGraph; each node reads the same evolving state dictionary and returns its output fields. It is not a set of autonomous LLM agents chatting with one another. Node roles are:

1. Property classification agent: combines supplied fields with deterministic extraction from the description, canonicalizes neighborhood aliases, and validates features.
2. Comparable retrieval agent: queries the hybrid retriever and returns comparable records plus filter/fallback metadata.
3. Valuation agent: predicts with the selected regression model and generates SHAP impacts (or the explicit feature-importance fallback); if prediction is unavailable, uses a comparable-sale median.
4. Risk/explainability agent: calculates confidence and uncertainty from completeness, similarity, comparable price spread, and fallback flags, then identifies safeguards.
5. Decision/insight agent: assembles the user-facing valuation, evidence, impacts, and caveats.

Communication is through typed shared state keys (item, features, comparables, retrieval, valuation, risk, result) in a fixed edge order. The graph is invoked by the valuation service. If LangGraph cannot initialize, the service calls the same stages sequentially as a deterministic fallback. Claude is a separate post-valuation judge; it is not one of these valuation graph nodes and cannot edit the estimate.

8. Runtime data flow

1. A user enters a description and any known features in the frontend.
2. FastAPI validates input bounds and rejects instruction-like query content.
3. The workflow extracts structured facts, then retrieves comparable properties with filters, semantic search, BM25, and RRF.
4. The prediction stage uses the trained model. If unavailable, a documented comparable-based fallback may be used.
5. Risk and explanation stages produce uncertainty, safeguards, evidence, and feature impacts.
6. The response is returned to the frontend and may be saved in SQLite.
7. Optional LLM-as-Judge and DeepEval evaluations run separately and cannot change price, range, or confidence.

See architecture diagram and API reference.

9. Data and artifacts

The training and retrieval input is data/AmesHousing.csv, OpenML dataset 43926. SalePrice is the prediction target. Model features are configured in the backend; GarageYrBlt is conditional on dataset availability. The source CSV, model bundle, ChromaDB index, and SQLite store have separate roles. Preparation, training, and embedding rebuild commands are documented in the README. See dataset details for provenance, inspected snapshot profile, preprocessing, and limitations.

10. API and frontend contract

The core request model accepts a natural-language query and optional structured property fields. /api/valuate returns the valuation and initial DeepEval pending state. /api/deepeval/evaluate evaluates the submitted property and returns the one GEval metric or an unavailable result. /api/search and /api/compare return comparable evidence; /api/what-if returns model-based scenario estimates; /api/feedback persists a rating when possible. Full endpoint and error details are in API documentation and its PDF. OpenAPI is available at /docs when the service is running.

11. Evaluation strategy

- DeepEval: one GEval metric, Property Recommendation Quality, threshold 0.6. It assesses whether the recommendation uses the input, grounds the estimate in historical comparables, identifies uncertainty, and avoids unsupported or causal claims. The API reports score, threshold verdict, and reason; if Claude or DeepEval fails, it reports evaluation unavailable without breaking valuation.
- RAGAS: not implemented. No RAGAS metrics or scores should be claimed for this project.
- LLM-as-Judge: a separate Claude explanation judge returns groundedness, relevance, completeness, evidence usage, reasoning quality, verdict, and unsupported claims. If not configured or unavailable, deterministic checks are labelled as such; they are not LLM scores.
- Regression metrics: training compares candidate regressors by holdout MAE; the training artifact also records MAE, RMSE, and R-squared (R?). These measure prediction error/fit on the configured split, not live-market accuracy.
- Retrieval evaluation: scripts/evaluateretrieval.py reports latency and number of retrieved items on a real dataset query. The project has no labelled relevance set, so it does not claim retrieval precision, recall, MAP, or nDCG.
- Automated tests: tests/testcore.py covers extraction, prompt-like query rejection, name-only rejection, and missing-information handling. tests/testretrieval.py covers normalized embeddings, RRF, BM25 ranking, hybrid search, and semantic fallback. tests/testdeepevalquality.py covers GEval adapter behavior, unavailable handling, endpoint result, and pending valuation state. tests/testllmjudge.py covers response parsing, configured Anthropic calls, and deterministic fallback. tests/testexplainability.py covers feature-importance fallback.

12. Challenges and trade-offs

- The Ames data is historical and geographically narrow; the model is not suitable for current market valuation or other regions without representative data and retraining.
- Structured row-level documents make retrieval auditable and avoid arbitrary text chunks, but they do not support full-text property reports. Long documents would need a chunking policy and deduplication.
- MiniLM and local Chroma reduce embedding-service cost and keep vectors local, with trade-offs in download time, local memory, and model quality compared with larger hosted encoders.
- Semantic retrieval handles paraphrases while BM25 is strong for exact tokens; RRF avoids calibrating incomparable raw scores but may not be optimal without labelled relevance judgments.
- Strict metadata filters improve comparability but can return too few records. Progressive relaxation improves coverage and is surfaced as a safeguard, at the cost of less exact matches.
- A tree-regression model is efficient for tabular features and explainable through SHAP, but its estimates reflect historical correlations and dataset bias. Feature impacts are descriptive model explanations, not causal effects.
- External Claude/DeepEval calls add latency, network dependence, and provider cost. Evaluations are optional and isolated from the price estimate; failures must remain visible rather than being represented as successful LLM judgments.
- Incomplete descriptions require median imputation. The UI surfaces incompleteness and uncertainty because imputation cannot recover facts that were never supplied.

13. Reliability and safeguards

- Input validation returns HTTP 422 for invalid values or unsupported property changes.
- Chroma/embedding/model availability is reported by health endpoints and deterministic retrieval or valuation fallbacks are used where implemented.
- Optional LLM and DeepEval failures are reported as unavailable or deterministic evaluation status; valuation remains available.
- Missing values are imputed with training medians where required.
- Comparables and uncertainty are historical evidence. Explanations avoid unsupported causal interpretation.
- The LLM receives calculated valuation context for explanation review and cannot author numeric prediction fields.

14. Local operation

Install backend dependencies, prepare/train data if artifacts need rebuilding, run the

embedding build script, start FastAPI, then start the Vite frontend. The README contains exact PowerShell commands. The environment file may configure the Anthropic key for optional judge functionality; embedding inference remains local and does not use a paid embedding API.

15. Verification

The repository includes backend tests under tests/. Run python -m pytest from the project root to exercise the implemented behavior. Rebuild the two PDF deliverables with python scripts/buildarchitecturepdf.py and python scripts/builddesigndocumentpdf.py.